

Лабораторная работа №9. Kubernetes

Краткое описание

В лабораторной работе №9 приложение перенесено в Kubernetes-окружение. Цель внедрения: запустить веб-приложение и зависимости через манифесты Kubernetes, проверить health checks, горизонтальное масштабирование API, работу асинхронных событий RabbitMQ при нескольких репликах и распределенную блокировку регистрации через Redis.

Используемые компоненты:

- ? FastAPI-приложение: основной HTTP API.
- ? MongoDB: документная база данных.
- ? Redis: кеш, JWT access-токенов, идемпотентность событий и распределенная блокировка.
- ? MinIO: объектное хранилище файлов.
- ? RabbitMQ: брокер сообщений для события регистрации пользователя.
- ? Mailpit: локальная SMTP-песочница для проверки приветственных писем.

Что добавлено в код приложения

app/main.py

Назначение: точка сборки FastAPI-приложения, регистрация роутеров и фоновых процессов.

Ключевые места:

- ? app/main.py:11 - подключен роутер health-эндпоинтов.
- ? app/main.py:47 - startup hook приложения.
- ? app/main.py:50-52 - попытка создать MongoDB-индексы; если MongoDB недоступна, приложение не падает, а /health/ready показывает not_ready.
- ? app/main.py:55-58 - попытка настроить RabbitMQ; если брокер временно недоступен, consumer переподключается в фоне.
- ? app/main.py:60 - запуск фонового RabbitMQ consumer.
- ? app/main.py:74 - подключение health router.
- ? app/main.py:78 - подключение /users alias для регистрации в сценарии Kubernetes.

app/modules/health/router.py

Назначение: health-эндпоинты для Kubernetes probes.

Ключевые места:

- ? app/modules/health/router.py:14 - роутер с префиксом /health.
- ? app/modules/health/router.py:25 - GET /health, общий статус процесса.
- ? app/modules/health/router.py:29-30 - GET /health/live, легкая liveness-проверка.
- ? app/modules/health/router.py:34-35 - GET /health/ready, readiness-проверка.
- ? app/modules/health/router.py:39 - проверка MongoDB через ping.
- ? app/modules/health/router.py:45 - проверка Redis через ping.
- ? app/modules/health/router.py:51 - проверка RabbitMQ через rabbitmq.ping().
- ? app/modules/health/router.py:56 - проверка MinIO через list_buckets().
- ? app/modules/health/router.py:69 - возврат 200, если все зависимости доступны, иначе 503.

app/common/cache.py

Назначение: общий Redis-клиент, кеширование и распределенная блокировка.

Ключевые места:

- ? app/common/cache.py:32 - проверка Redis через ping.
- ? app/common/cache.py:79 - метод acquire_lock.
- ? app/common/cache.py:85 - атомарное получение lock через Redis SET key value EX ttl NX.
- ? app/common/cache.py:91 - метод release_lock.
- ? app/common/cache.py:93-99 - атомарное снятие lock через Lua: удалить ключ можно только если значение совпадает с lock_id.

app/modules/auth/service.py

Назначение: регистрация пользователя, выдача токенов, публикация события регистрации.

Ключевые места:

- ? app/modules/auth/service.py:130 - метод register.
- ? app/modules/auth/service.py:132 - формирование ключа блокировки wp:locks:auth:register:<email>.
- ? app/modules/auth/service.py:133 - попытка получить Redis lock.
- ? app/modules/auth/service.py:134-135 - если регистрация этого email уже идет, возвращается конфликт.
- ? app/modules/auth/service.py:144 - публикация события user.registered в RabbitMQ.
- ? app/modules/auth/service.py:146-147 - обработка гонки уникального email на уровне MongoDB.
- ? app/modules/auth/service.py:149 - освобождение Redis lock.

app/common/queue/rabbitmq.py

Назначение: единый слой работы с RabbitMQ.

Ключевые места:

- ? app/common/queue/rabbitmq.py:37 - декларация topology RabbitMQ.
- ? app/common/queue/rabbitmq.py:38-45 - durable direct exchanges app.events и app.dlx.
- ? app/common/queue/rabbitmq.py:47-54 - очередь wp.auth.user.registered с dead-letter routing.
- ? app/common/queue/rabbitmq.py:61-66 - DLQ wp.auth.user.registered.dlx.
- ? app/common/queue/rabbitmq.py:76 - метод ping для readiness.
- ? app/common/queue/rabbitmq.py:94 - publisher confirms.
- ? app/common/queue/rabbitmq.py:102 - persistent-сообщения через delivery_mode=2.
- ? app/common/queue/rabbitmq.py:132 - некорректный JSON отправляется в DLQ через nack(requeue=False).
- ? app/common/queue/rabbitmq.py:157 - ack.
- ? app/common/queue/rabbitmq.py:160-163 - nack.

app/modules/users/router.py

Назначение: alias POST /users для сценария лабораторной работы №9.

Ключевые места:

- ? app/modules/users/router.py:13 - роутер с тегом Users.
- ? app/modules/users/router.py:16-18 - описание POST /users.
- ? app/modules/users/router.py:25 - вызов AuthService.register.
- ? app/modules/users/router.py:26 - установка auth cookies после регистрации.

Kubernetes-манифесты

k8s/00-namespace.yaml

Назначение: отдельное пространство имен wp - labs.

Ключевые места:

- ? k8s/00-namespace.yaml:2 - ресурс Namespace.
- ? k8s/00-namespace.yaml:4 - имя namespace wp - labs.

k8s/02-mongodb

Назначение: MongoDB в Kubernetes.

Файлы:

- ? k8s/02-mongodb/secret.yaml - логин, пароль и имя БД.
- ? k8s/02-mongodb/service.yaml - внутренний сервис mongo:27017.
- ? k8s/02-mongodb/statefulset.yaml - MongoDB как StatefulSet.

Ключевые места:

- ? k8s/02-mongodb/statefulset.yaml:2 - StatefulSet.
- ? k8s/02-mongodb/statefulset.yaml:7 - serviceName: mongo.
- ? k8s/02-mongodb/statefulset.yaml:8 - одна реплика.
- ? k8s/02-mongodb/statefulset.yaml:28-42 - переменные окружения из Secret.
- ? k8s/02-mongodb/statefulset.yaml:43 - startupProbe.
- ? k8s/02-mongodb/statefulset.yaml:54 - livenessProbe.
- ? k8s/02-mongodb/statefulset.yaml:65 - readinessProbe.
- ? k8s/02-mongodb/statefulset.yaml:84-88 - постоянное хранилище MongoDB.

k8s/03-redis

Назначение: Redis для кеша, JTI, идемпотентности и distributed lock.

Файлы:

- ? k8s/03-redis/secret.yaml - пароль Redis.
- ? k8s/03-redis/deployment.yaml - Redis deployment.
- ? k8s/03-redis/service.yaml - внутренний сервис redis:6379.

Ключевые места:

- ? k8s/03-redis/deployment.yaml:2 - Deployment.
- ? k8s/03-redis/deployment.yaml:20-22 - запуск Redis с --requirepass.
- ? k8s/03-redis/deployment.yaml:27-31 - пароль из Secret.
- ? k8s/03-redis/deployment.yaml:32 - livenessProbe.
- ? k8s/03-redis/deployment.yaml:41 - readinessProbe.

k8s/04-minio

Назначение: объектное хранилище файлов.

Файлы:

- ? k8s/04-minio/secret.yaml - root user и password.
- ? k8s/04-minio/service.yaml - внутренние порты 9000 и 9001.
- ? k8s/04-minio/statefulset.yaml - MinIO как StatefulSet.

Ключевые места:

- ? k8s/04-minio/statefulset.yaml:2 - StatefulSet.
- ? k8s/04-minio/statefulset.yaml:27 - запуск MinIO server и console.
- ? k8s/04-minio/statefulset.yaml:34-43 - credentials из Secret.

- ? k8s/04-minio/statefulset.yaml:44 - livenessProbe.
- ? k8s/04-minio/statefulset.yaml:50 - readinessProbe.
- ? k8s/04-minio/statefulset.yaml:64-68 - постоянное хранилище MinIO.

k8s/05-rabbitmq

Назначение: RabbitMQ broker с Management UI.

Файлы:

- ? k8s/05-rabbitmq/secret.yaml - пользователь, пароль и Erlang cookie.
- ? k8s/05-rabbitmq/service.yaml - порты 5672 и 15672.
- ? k8s/05-rabbitmq/statefulset.yaml - RabbitMQ как StatefulSet.

Ключевые места:

- ? k8s/05-rabbitmq/statefulset.yaml:2 - StatefulSet.
- ? k8s/05-rabbitmq/statefulset.yaml:20-25 - init container для прав на volume.
- ? k8s/05-rabbitmq/statefulset.yaml:35-55 - настройки RabbitMQ из Secret.
- ? k8s/05-rabbitmq/statefulset.yaml:57 - livenessProbe.
- ? k8s/05-rabbitmq/statefulset.yaml:64 - readinessProbe.
- ? k8s/05-rabbitmq/statefulset.yaml:71 - startupProbe.
- ? k8s/05-rabbitmq/statefulset.yaml:85-89 - постоянное хранилище RabbitMQ.

k8s/06-api

Назначение: запуск FastAPI-приложения в Kubernetes.

Файлы:

- ? k8s/06-api/configmap.yaml - нечувствительная конфигурация.
- ? k8s/06-api/secret.yaml - чувствительные переменные окружения.
- ? k8s/06-api/deployment.yaml - deployment API.
- ? k8s/06-api/service.yaml - внутренний сервис api:4200.

Ключевые места:

- ? k8s/06-api/configmap.yaml:13-16 - MongoDB host/port/db.
- ? k8s/06-api/configmap.yaml:18-21 - Redis host/port/db/prefix.
- ? k8s/06-api/configmap.yaml:26-29 - MinIO endpoint и bucket.
- ? k8s/06-api/configmap.yaml:33-40 - RabbitMQ exchange, DLQ, retries.
- ? k8s/06-api/configmap.yaml:42-47 - SMTP через Mailpit.
- ? k8s/06-api/secret.yaml:10-22 - MongoDB, Redis, MinIO, RabbitMQ и SMTP secret values.
- ? k8s/06-api/deployment.yaml:2 - Deployment.
- ? k8s/06-api/deployment.yaml:10 - базово две реплики API.
- ? k8s/06-api/deployment.yaml:35-38 - подключение ConfigMap и Secret.
- ? k8s/06-api/deployment.yaml:39 - startupProbe.
- ? k8s/06-api/deployment.yaml:46 - livenessProbe на /health/live.
- ? k8s/06-api/deployment.yaml:57 - readinessProbe на /health/ready.
- ? k8s/06-api/service.yaml:2 - Service.
- ? k8s/06-api/service.yaml:11-14 - порт 4200.

k8s/07-mailpit

Назначение: локальная SMTP-песочница для проверки отправки писем.

Ключевые места:

- ? k8s/07-mailpit/deployment.yaml:2 - Deployment.
- ? k8s/07-mailpit/deployment.yaml:22-25 - порты SMTP 1025 и Web UI 8025.
- ? k8s/07-mailpit/deployment.yaml:26 - readinessProbe.
- ? k8s/07-mailpit/deployment.yaml:32 - livenessProbe.
- ? k8s/07-mailpit/service.yaml:13-18 - сервисные порты SMTP и HTTP.

Сценарии тестирования

1. Сборка Docker-образа

```
cd D:\fastapi-newyear
docker build -t wp-labs/api:1.0.0 .
```

2. Деплой Kubernetes-манифестов

```
kubectl apply -f k8s/00-namespace.yaml
kubectl apply -f k8s/02-mongodb/
kubectl apply -f k8s/03-redis/
kubectl apply -f k8s/04-minio/
kubectl apply -f k8s/05-rabbitmq/
kubectl apply -f k8s/07-mailpit/
kubectl apply -f k8s/06-api/
```

3. Проверка состояния кластера

```
kubectl get all -n wp-labs
kubectl get pods -n wp-labs -o wide
kubectl get endpoints -n wp-labs
```

Ожидаемо: поды api, mongo, redis, minio, rabbitmq, mailpit находятся в статусе Running.

4. Проброс портов

```
kubectl port-forward svc/api 4301:4200 -n wp-labs
```

Во втором терминале:

```
kubectl port-forward svc/mailpit 8026:8025 -n wp-labs
```

Адреса:

- ? API: http://localhost:4301
- ? Swagger: http://localhost:4301/api/docs
- ? Mailpit: http://localhost:8026

5. Проверка health endpoints

```
curl.exe http://localhost:4301/health
curl.exe http://localhost:4301/health/live
curl.exe http://localhost:4301/health/ready
```

Ожидаемый readiness:

```
{
  "status": "ready",
  "checks": {
    "mongo": {"status": "ok"},
    "redis": {"status": "ok"},
    "rabbitmq": {"status": "ok"},
    "minio": {"status": "ok"}
  }
}
```

6. Проверка Swagger

Открыть:

```
http://localhost:4301/api/docs
```

Проверить наличие эндпоинтов:

```
? /health
? /health/live
? /health/ready
? /users
? /auth/register
? /auth/login
? /items
? /files
? /profile
```

7. Проверка регистрации и RabbitMQ

```
$body = @{
  email = "lab9.test@example.com"
  password = "Password123!"
  displayName = "Lab9 User"
} | ConvertTo-Json

curl.exe -i `
  -H "Content-Type: application/json" `
  --data-binary $body `
  http://localhost:4301/users
```

Ожидаемо: HTTP/1.1 201 Created.

Проверить логи:

```
kubectl logs -l app=api -n wp-labs --tail=120
```

Ожидаемые строки:

```
Published RabbitMQ event
Received RabbitMQ event
Sending welcome email
welcome email sent
```

Проверить очереди RabbitMQ:

```
kubectl exec statefulset/rabbitmq -n wp-labs -- rabbitmqctl list_queues name messages_ready
messages_unacknowledged
```

Ожидаемо:

```
wp.auth.user.registered      0 0
wp.auth.user.registered.dlq  0 0
```

8. Проверка письма

Открыть:

```
http://localhost:8026
```

Ожидаемо: письмо с темой Welcome to Holiday Prep.

9. Проверка горизонтального масштабирования

```
kubectl scale deployment/api --replicas=4 -n wp-labs
kubectl rollout status deployment/api -n wp-labs
```

```
kubectl get pods -n wp-labs -l app=api
```

Ожидаемо: 4/4 pod API в статусе Running.

После масштабирования повторить регистрацию через /users. Проверить, что:

- ? пользователь создан;
- ? письмо пришло один раз;
- ? очередь RabbitMQ снова пустая.

10. Проверка DLQ

Остановить Mailpit:

```
kubectl scale deployment/mailpit --replicas=0 -n wp-labs
```

Создать пользователя:

```
$body = @{
  email = "lab9.dlq@example.com"
  password = "Password123!"
  displayName = "DLQ User"
} | ConvertTo-Json

curl.exe -i `
  -H "Content-Type: application/json" `
  --data-binary $body `
  http://localhost:4301/users
```

Проверить очереди:

```
kubectl exec statefulset/rabbitmq -n wp-labs -- rabbitmqctl list_queues name messages_ready
messages_unacknowledged
```

Ожидаемо после retry: в wp.auth.user.registered.dlq появится сообщение.

Вернуть Mailpit:

```
kubectl scale deployment/mailpit --replicas=1 -n wp-labs
kubectl rollout status deployment/mailpit -n wp-labs
```

Очистить DLQ после теста:

```
kubectl exec statefulset/rabbitmq -n wp-labs -- rabbitmqctl purge_queue wp.auth.user.
registered.dlq
```

11. Проверка распределенной блокировки

Идея проверки: при нескольких репликах API отправить несколько одинаковых регистраций на один email почти одновременно.

Пример:

```
$body = @{
  email = "lab9.lock@example.com"
  password = "Password123!"
  displayName = "Lock User"
} | ConvertTo-Json -Compress

1..5 | ForEach-Object {
  Start-Job -ScriptBlock {
    param($json)
    curl.exe -s -i -H "Content-Type: application/json" --data-binary $json http://localhost:
4301/users
  } -ArgumentList $body
}

Get-Job | Receive-Job -Wait
Remove-Job *
```

Ожидаемо: один запрос создаст пользователя, остальные получают конфликт. Это подтверждает, что критическая секция регистрации защищена Redis lock.

Ответы на контрольные вопросы

1. Что такое Kubernetes и какую проблему он решает?

Kubernetes - это система оркестрации контейнеров. Он решает проблему запуска, обновления, масштабирования и восстановления контейнеризированных приложений. Вместо ручного запуска контейнеров Kubernetes поддерживает желаемое состояние: сколько реплик должно работать, какие порты открыть, какие переменные передать, когда перезапустить контейнер, какие поды готовы принимать трафик.

2. В чем разница между Pod, Deployment и StatefulSet? Когда использовать каждый?

Pod - минимальная единица запуска в Kubernetes. В нем находится один или несколько контейнеров с общей сетью и volume.

Deployment - контроллер для stateless-приложений. Он управляет ReplicaSet, rolling update, rollback и масштабированием. Используется для API, frontend, background workers без уникального состояния на конкретной реплике.

StatefulSet - контроллер для stateful-сервисов. Он дает стабильные имена pod-ов, порядок запуска и привязку volume к конкретной реплике. Используется для MongoDB, RabbitMQ, MinIO и похожих сервисов.

3. Какую роль играет Service в архитектуре Kubernetes? Перечислите типы и их отличия.

Service дает стабильную сетевую точку доступа к группе pod-ов. Pod-ы могут пересоздаваться и менять IP, а Service остается постоянным DNS-именем.

Типы Service:

- ? ClusterIP - доступ только внутри кластера. Используется по умолчанию.
- ? NodePort - открывает порт на каждой node.
- ? LoadBalancer - создает внешний балансировщик в облачной инфраструктуре.
- ? ExternalName - DNS-alias на внешний адрес.
- ? Headless Service (clusterIP: None) - не балансирует трафик, а отдает DNS-записи pod-ов; полезен для StatefulSet.

4. Как представлены секреты в Kubernetes? Какие меры безопасности необходимо учитывать при работе с ними?

Секреты представлены ресурсом Secret. В манифестах можно использовать stringData или base64-поля в data. В pod-ы они попадают как переменные окружения или файлы.

Важно учитывать:

- ? Secret не является полноценным шифрованием сам по себе.
- ? В production нужно включать encryption at rest для etcd.
- ? Доступ к Secret нужно ограничивать через RBAC.
- ? Нельзя коммитить реальные production-пароли в репозиторий.
- ? Нельзя логировать секреты.
- ? Для production лучше использовать External Secrets, Vault или облачный Secret Manager.

5. Что такое горизонтальное и вертикальное масштабирование? Какой тип поддерживает kubectl scale?

Горизонтальное масштабирование - увеличение количества реплик приложения. Например, API было 2 pod-а, стало 4.

Вертикальное масштабирование - увеличение ресурсов одной реплики: CPU, RAM, limits/requests.

`kubectl scale` выполняет горизонтальное масштабирование, то есть меняет количество replicas.

6. В чем разница между `livenessProbe` и `readinessProbe`? Почему их не следует делать идентичными?

`livenessProbe` отвечает на вопрос: процесс жив или завис так, что контейнер нужно перезапустить. Она должна быть легкой.

`readinessProbe` отвечает на вопрос: можно ли отправлять трафик в этот pod прямо сейчас. Она может проверять зависимости: БД, Redis, RabbitMQ, MinIO.

Их не стоит делать идентичными, потому что временная недоступность БД не всегда означает, что процесс надо перезапускать. В таком случае pod должен быть исключен из балансировки через `readiness`, но не обязательно уничтожен через `liveness`.

7. Как Kubernetes определяет, что pod недоступен для получения трафика?

Kubernetes смотрит на состояние `readiness`. Если `readinessProbe` возвращает ошибку или код не из диапазона успешных ответов, pod получает состояние `Ready=False`. Service перестает отправлять на него пользовательский трафик, пока `readiness` снова не станет успешной.

8. Что такое Namespace и зачем он нужен в многопользовательском кластере?

Namespace - логическое пространство имен внутри кластера. Он помогает разделять окружения, команды и ресурсы. В многопользовательском кластере namespace нужен для изоляции имен, RBAC-доступов, quotas и удобного управления ресурсами.

В этой работе используется namespace `wp-labs`.

9. Почему масштабирование stateful-сервисов через репликацию pod-ов не гарантирует консистентность данных?

Stateful-сервисы хранят состояние: данные, очереди, индексы, файловые объекты. Если просто увеличить количество pod-ов MongoDB, RabbitMQ или MinIO, Kubernetes создаст больше контейнеров, но не настроит автоматически репликацию данных, leader election, quorum, синхронизацию журналов и восстановление консистентности.

Для корректного масштабирования stateful-сервисов нужны специальные cluster modes и правила конкретного продукта: MongoDB Replica Set, RabbitMQ cluster/quorum queues, MinIO distributed mode и так далее.

10. Какие условия должны выполняться для корректной работы распределенной блокировки в Redis?

Для корректной distributed lock нужны условия:

- ? Получение lock должно быть атомарным: `SET key value NX EX ttl`.
- ? У каждого владельца lock должен быть уникальный `lock_id`.
- ? Lock должен иметь TTL, чтобы не зависнуть навсегда при падении процесса.
- ? Снятие lock должно быть атомарным и проверять владельца: удалить ключ можно только если значение равно текущему `lock_id`.
- ? Критическая секция должна быть короче TTL или TTL нужно безопасно продлевать.
- ? Код должен корректно освобождать lock в `finally`.
- ? Нужно понимать ограничения Redis lock при сетевых partition и failover; для production-критичных сценариев требуется более строгий дизайн.

В проекте получение lock реализовано в `app/common/cache.py:79-89`, снятие lock - в `app/common/cache.py:91-101`, применение при регистрации - в `app/modules/auth/service.py:132-149`.